

TECHNICAL DOCUMENT

Backend Design Requirements

Development approach, database design, technology stack, API design, and performance strategy for the Product Management REST API.

PROJECT

Product Management API

RUNTIME

.NET 10 / ASP.NET Core

DATE

August 2026

Table of Contents

1 Approach to the Requirement

1.1 Architecture Decision — Clean Architecture

1.2 Development Process Steps

1.3 Project & Layer Structure

2 Database Design

2.1 SQL vs NoSQL Decision

2.2 Entity Relationship Overview

2.3 Schema Design Highlights

2.4 Supporting New Product Features & Attributes

3 Technology Stack Components

3.1 ORM Framework

3.2 Validation

3.3 Full Stack Summary

4 API & Data Handling

4.1 Input Processing Pipeline

4.2 API Design Principles

4.3 Error Handling & ProblemDetails

4.4 API Endpoint Reference

5 Performance

5.1 Caching Strategy

5.2 Concurrency Control

5.3 Database Query Optimisation

1

Approach to the Requirement

1.1 Architecture Decision — Clean Architecture

The backend adopts **Clean Architecture** (also known as Onion Architecture). The core rule is that dependencies always point inward — outer layers know about inner layers, never the reverse. This is machine-enforced by a dedicated `ProductManagement.ArchitectureTests` project using `NetArchTest.Rules`.

Why Clean Architecture?

- Domain and business logic are completely isolated from framework concerns
- Swapping the database, cache, or web framework requires changes only in the outer layers
- Architecture rules are verified by automated tests, not just documented
- Each layer is independently unit-testable

Enforced dependency rules

- **Domain** — zero external dependencies
- **Application** — depends only on Domain
- **Infrastructure** — depends on Application + Domain
- **Api** — depends on Application only (never Infrastructure directly)

1.2 Development Process Steps

1

Domain modelling

Define entities, value objects, domain exceptions, and business rules in the `Domain` project. Entities use private setters and static factory methods (`Product.Create( ... )`) to enforce invariants at construction time. No external dependencies are introduced at this stage.

2

Application layer — interfaces and handlers

Define repository and service interfaces (`IProductRepository` , `ICacheService`) in `Application` . Implement CQRS-style handler classes (no MediatR) and FluentValidation validators co-located with their request records. The application layer never references EF Core or Redis directly.

3

Database design & EF Core configuration

Implement `IEntityTypeConfiguration<T>` classes mapping entities to PostgreSQL tables. Configure indexes, constraints, computed columns (`tsvector`), and the `xmin` optimistic concurrency token. Generate and review EF Core migrations.

4

Infrastructure — repositories, cache, storage

Implement repository interfaces using EF Core and the `ProductManagementDbContext`. Implement `RedisCacheService` (StackExchange.Redis), `LocalFileStorageService` for image uploads, and `UnitOfWork` (catches PostgreSQL constraint violations and converts to domain exceptions).

5

API layer — controllers, exception handler, middleware

Build ASP.NET Core controllers delegating to handlers. Implement `GlobalExceptionHandler` that maps every domain and application exception to an RFC-9457 `ProblemDetails` response. Configure CORS, static file serving, Swagger, and Serilog structured logging.

6

Testing — unit, integration, architecture

Write unit tests for domain entities and application handlers (NSubstitute mocks). Write integration tests using `WebApplicationFactory` against a real PostgreSQL test database (Respawn resets between tests). Architecture tests enforce layer dependency rules with NetArchTest. Seed 5 000 products using Bogus for realistic load.

7

Containerisation & seed data

Two-stage Dockerfile (SDK build → ASP.NET runtime). Docker Compose brings up postgres:17, redis:7, api, and web in one command. On first boot in non-Production, EF Core migrations run automatically and `DbInitializer` seeds 5 000 products (idempotent — skips if data exists).

1.3 Project & Layer Structure

```

backend/ ├── src/ | ├── ProductManagement.Domain/ # entities, enums, domain
exceptions | ├── ProductManagement.Application/ # handlers, validators,
interfaces, DTOs | ├── ProductManagement.Infrastructure/ # EF Core, Redis, file
storage, seeder | └── ProductManagement.Api/ # controllers, middleware, Program.cs
└── tests/ ├── ProductManagement.UnitTests/ # domain + handler unit tests
(NSubstitute) ├── ProductManagement.IntegrationTests/ # HTTP-level tests
(WebApplicationFactory) └── ProductManagement.ArchitectureTests/ # dependency rule
enforcement (NetArchTest)

```

2

Database Design

2.1 SQL vs NoSQL Decision

PostgreSQL 17 (relational SQL) was chosen. The decision rests on four pillars:

Requirement	Why PostgreSQL wins
Relational integrity	Products, categories, variations, promotions, and items have real FK relationships. Referential integrity must be enforced at the database level, not the application.
Flexible attributes	PostgreSQL's native <code>jsonb</code> column stores arbitrary product attributes (material, colour, etc.) with GIN indexing and <code>@></code> containment queries — giving NoSQL-style flexibility inside a relational schema.
Full-text search	Built-in <code>tsvector</code> / <code>tsquery</code> with stored computed columns and GIN indexes delivers ranked search without a separate search service. Trigram (<code>pg_trgm</code>) provides fuzzy fallback.
Optimistic concurrency	PostgreSQL exposes the <code>xmin</code> system column — a free, per-row version token that EF Core uses for concurrency checks with zero schema changes.

A document store (MongoDB) would handle flexible attributes well but loses referential integrity, ACID transactions, and native FTS. A key-value store (Redis) is used as a *cache layer* on top of PostgreSQL, not as the primary store.

2.2 Entity Relationship Overview

Table	Purpose	Key relationships
categories	Product taxonomy (self-referential tree)	parent_category_id → categories.id (RESTRICT)
products	Core product record	category_id → categories.id (RESTRICT)
product_items	SKU-level variant (price + stock)	product_id → products.id (RESTRICT)
variation	Dimension name (e.g. "Size", "Colour")	category_id → categories.id (RESTRICT)
variation_option	Dimension value (e.g. "M", "Blue")	variation_id → variation.id (CASCADE)

Table	Purpose	Key relationships
product_configuration	Maps item ↔ variation options	product_item_id (CASCADE), variation_option_id (RESTRICT)
promotion	Discount with date range and rate	—
promotion_category	Applies a promotion to a category	promotion_id (CASCADE), category_id (RESTRICT)

2.3 Schema Design Highlights

Case-insensitive slugs — citext

The `slug` column on both `products` and `categories` uses PostgreSQL's `citext` extension. Uniqueness checks are automatically case-insensitive without requiring `LOWER()` function indexes or application-level normalisation.

Full-text search — stored computed tsvector

```
-- Computed column on products table (maintained by Postgres)
search_vector tsvector GENERATED ALWAYS AS (
  setweight(to_tsvector('english', coalesce(name, '')),      'A') ||
  setweight(to_tsvector('english', coalesce(brand, '')),    'B') ||
  setweight(to_tsvector('english', coalesce(description, '')), 'C')
) STORED
```

GIN index on `search_vector` makes FTS queries instant even at 5 000+ products. Name ranks highest (weight A), brand second (B), description third (C). When FTS returns zero hits, a trigram similarity query (`gin_trgm_ops` index on `name`) provides fuzzy fallback.

Optimistic concurrency — xmin system column

PostgreSQL increments `xmin` on every row update at the storage engine level. EF Core maps it as a shadow concurrency token — no application-managed version column is needed on `products` .

Database-level constraints

Table	Constraint	Rule
product_items	ck_product_items_price	price >= 0
product_items	ck_product_items_qty	qty_in_stock >= 0
promotion	ck_promotion_discount_rate	discount_rate > 0 AND <= 1
promotion	ck_promotion_dates	end_date >= start_date

2.4 Supporting New Product Features & Attributes

Flexible attributes via `jsonb`

The `products.attributes` column stores arbitrary key-value metadata as `jsonb`. Adding a new attribute (e.g. `{"warranty": "2 years"}`) requires zero schema changes — no migration, no new column. A GIN index with `jsonb_path_ops` allows containment queries like `attributes @> '{"material": "Cotton"}'` to run efficiently.

Variation / Option / Configuration model

New product dimensions (size, colour, material, etc.) are modelled as rows in the `variation` and `variation_option` tables, scoped per category. A new dimension never requires a schema change — adding "Fit" as a variation for the "Clothing" category is a data operation, not a migration. The `product_configuration` join table then maps each `product_item` to its chosen option values.

```
-- Example: adding a new "Fit" variation for Clothing (no migration needed)
INSERT INTO variation (category_id, name) VALUES (5, 'Fit');
-- Options
INSERT INTO variation_option (variation_id, value) VALUES (10, 'Slim'), (10, 'Regular')
-- Link to a product item
INSERT INTO product_configuration (product_item_id, variation_option_id) VALUES (42, 1
```

Promotion system

Promotions are a separate entity with `discount_rate`, date range, and a many-to-many link to categories via `promotion_category`. New promotional mechanics (flash sales, tiered discounts) extend this model without touching products.

3

Technology Stack Components

3.1 ORM Framework — Entity Framework Core 10

EF Core 10 with the **Npgsql provider** is the sole ORM. Entity mapping uses `IEntityTypeConfiguration<T>` classes — never data annotations on domain models, keeping the Domain project free of infrastructure concerns.

EF Core Feature	How it is used
Fluent API configuration	All table names, column types, indexes, and constraints are declared in <code>*Configuration.cs</code> classes
Shadow properties	<code>xmin</code> (concurrency token) and <code>SearchVector</code> (computed tsvector) are shadow properties — invisible to the domain entity
Owned / navigation	<code>Product.Items</code> loads via <code>.Include().ThenInclude()</code> ; read-only collections enforce aggregate encapsulation
ExecuteUpdateAsync	Stock adjustments use a bulk UPDATE without loading the entity — single round-trip, atomic conditional update
EF Functions	<code>EF.Functions.WebSearchToTsQuery</code> , <code>TrigramsSimilarity</code> , <code>JsonContains</code> translated to native Postgres SQL
Migrations	Two migrations: <code>InitialCreate</code> and <code>ErdAlignment</code> (dropped old variant table, added variation/promotion model)
Auto-migrate on startup	<code>db.Database.MigrateAsync()</code> runs at boot in non-Production — zero-downtime schema updates in Docker

3.2 Validation — FluentValidation 11

Every command/request record has a paired `AbstractValidator<T>` co-located in the same file. Validators are registered via `AddValidatorsFromAssembly` and invoked by handlers before any business logic runs.

```
// Example: CreateProductRequestValidator
public sealed class CreateProductRequestValidator : AbstractValidator<CreateProductRequest>
{
    public CreateProductRequestValidator()
    {
        RuleFor(x => x.Name).NotEmpty().MaxLength(200);
        RuleFor(x => x.Slug).NotEmpty().MaxLength(200).Matches(@"^[a-z0-9-]+$");
    }
}
```

```
        RuleFor(x => x.Attributes)
            .Must(json => json.Length <= 8000).WithMessage("8 000 chars max")
            .Must(BeValidJson).WithMessage("must be valid JSON");
        RuleForEach(x => x.Items).ChildRules(v => {
            v.RuleFor(x => x.Sku).NotEmpty().MaximumLength(64);
            v.RuleFor(x => x.Price).GreaterThanOrEqualTo(0);
            v.RuleFor(x => x.QtyInStock).GreaterThanOrEqualTo(0);
        });
    }
}
```

A `ValidationException` from FluentValidation is caught by `GlobalExceptionHandler` and serialised as a `400 Bad Request` with per-field error details in the `ProblemDetails` body — the client receives `propertyName` and `errorMessage` for each failing field.

3.3 Full Stack Summary

Category	Technology	Version
Runtime	.NET / ASP.NET Core	10.0
ORM	Entity Framework Core + Npgsql provider	10.0
Primary database	PostgreSQL	17
Cache	Redis (StackExchange.Redis)	7 / 2.8
Validation	FluentValidation	11.11
API docs	Swashbuckle / Swagger	7.2
Logging	Serilog (structured JSON)	9.0
Seed data	Bogus (deterministic fake data)	35.6
Containerisation	Docker + Docker Compose	—
Test framework	xUnit + FluentAssertions	2.9 / 7.0
Mocking	NSubstitute	5.3
Integration testing	Microsoft.AspNetCore.Mvc.Testing + Respawn	10.0 / 6.2
Architecture testing	NetArchTest.Rules	1.3

Category	Technology	Version
Code coverage	coverlet	6.0

4

API & Data Handling

4.1 Input Processing Pipeline

Every inbound request flows through the same pipeline before any database work occurs:

```

HTTP Request
  |
  ▼
GlobalExceptionHandler (UseExceptionHandler - outermost catch-all)
  |
  ▼
Controller → reads body / headers / query params → maps to request record
  |
  ▼
Handler → IValidator<T>.ValidateAndThrowAsync() → ValidationException if invalid
                                                (→ 400 ProblemDetails with f
  |
  ▼
Repository / Domain method → DomainException if business rule violated
                             (→ 400 / 404 / 409 via GlobalExceptionHandler)
  |
  ▼
UnitOfWork.SaveChangesAsync() → catches PostgresException 23505 (unique violation)
                               converts to DuplicateSkuException / DuplicateSlugEx
  |
  ▼
Response DTO → serialised as JSON → HTTP 200 / 201 / 204
  
```

No MediatR — plain handler injection

Handlers are registered as `Scoped` services and injected directly into controllers. This keeps the pipeline explicit and debuggable without the overhead of a mediator bus. Pipeline cross-cutting concerns (validation, caching) are handled inside each handler rather than via pipeline behaviours.

4.2 API Design Principles

REST with versioned routes

All endpoints are prefixed with `/api/v1/`. Resources are noun-based. Actions that don't map cleanly to CRUD use a sub-resource verb path (e.g. `POST /product-items/{id}/inventory/adjust`).

Cursor-based pagination

The product list endpoint uses cursor pagination instead of `OFFSET`. An `OFFSET N` scan must skip N rows on every page — performance degrades linearly as pages grow. Cursors encode the last-seen row's sort key, making every page fetch $O(\log n)$ via the index regardless of page number.

Two cursor formats are used depending on the query path:

- **Browse path** (no search): cursor = `(createdAt, id)` — stable insertion-order pagination
- **Search path** (with `q=`): cursor = `(rank, id)` — stable relevance-order pagination

Cursors are base64-encoded JSON. An invalid cursor string returns `400 Bad Request`.

ETag / If-Match for optimistic concurrency

`GET /products/{id}` returns an `ETag: "xmin_value"` header. `PUT /products/{id}` requires the client to echo it back as `If-Match: "xmin_value"`. A missing or stale header always returns `409 Conflict`.

Idempotency-Key for stock adjustments

`POST /product-items/{id}/inventory/adjust` accepts an optional `Idempotency-Key` header. If present, the result is stored in Redis for 5 minutes. Retrying with the same key within the window returns the cached result without touching stock — safe for retry-on-timeout scenarios in distributed systems.

4.3 Error Handling & ProblemDetails

`GlobalExceptionHandler` implements `ExceptionHandler` (ASP.NET Core's built-in exception handler interface). Every exception type is mapped to an RFC-9457 `ProblemDetails` response with a consistent JSON shape.

Exception	HTTP Status	Notes
<code>ValidationException</code>	400	Includes per-field <code>fieldErrors[]</code> array
<code>DomainException</code>	400	Business rule violation
<code>InvalidImageException, InvalidCursorException</code>	400	Bad input format
<code>EntityNotFoundException, NoImageSetException</code>	404	Resource not found
<code>DuplicateSkuException, DuplicateSlugException</code>	409	Unique constraint violation
<code>CategoryHasActiveProductsException</code>	409	Referential integrity business rule
<code>ConcurrencyConflictException, DbUpdateConcurrencyException</code>	409	Stale ETag or version mismatch
<code>AdjustStockResult</code> (insufficient stock)	409	Returns <code>availableQuantity</code> in body
All others	500	TracedId logged via Serilog; opaque message to client

4.4 API Endpoint Reference

Method	Route	Purpose
GET	/api/v1/products	List with search, filters, cursor pagination
GET	/api/v1/products/{id}	Get by id — sets ETag header
GET	/api/v1/products/slug/{slug}	Get by slug — sets ETag header
POST	/api/v1/products	Create product with optional items
PUT	/api/v1/products/{id}	Update — requires If-Match header
DELETE	/api/v1/products/{id}	Delete product
POST	/api/v1/products/{id}/image	Upload product image (multipart)
DELETE	/api/v1/products/{id}/image	Remove product image
GET	/api/v1/products/{id}/items	List product items (variants)
POST	/api/v1/products/{id}/items	Add product item
PATCH	/api/v1/product-items/{id}	Update item — requires If-Match (version)
DELETE	/api/v1/product-items/{id}	Soft-delete product item
POST	/api/v1/product-items/{id}/inventory/adjust	Atomic stock adjustment
GET	/api/v1/categories	List categories
POST	/api/v1/categories	Create category
PUT	/api/v1/categories/{id}	Update category
DELETE	/api/v1/categories/{id}	Delete (blocked if has active products)
GET	/api/v1/categories/{id}/variations	List variations for a category
POST	/api/v1/categories/{id}/variations	Create variation
POST	/api/v1/variations/{id}/options	Add option to a variation
GET	/api/v1/promotions	List promotions
POST	/api/v1/promotions	Create promotion

Method	Route	Purpose
POST	/api/v1/promotions/{id}/categories/{catId}	Apply promotion to a category

5

Performance

5.1 Caching Strategy — Redis

Redis is used as a look-aside cache via the `ICacheService` abstraction (StackExchange.Redis, singleton connection). All values are serialised with `System.Text.Json`.

Cache key scheme

Key	TTL	Content
<code>product:{id}</code>	10 min	Full ProductDto + xmin
<code>products:list:version</code>	Persistent integer	Monotonic write counter
<code>products:list:v{n}:{queryHash}</code>	60 sec	Paginated list result (version-stamped)
<code>categories:list</code>	Standard TTL	Full category list
<code>stock-adjust:{idempotencyKey}</code>	5 min	AdjustStockResult for idempotency

Version-stamped list cache invalidation

Rather than enumerating all cached list keys when a product is written (which would require a Redis SCAN), a single integer counter `products:list:version` is incremented on every create or update. List cache keys embed the current version: `products:list:v42:a3f8c1 ...`. After a write, the counter becomes 43 — all v42 keys are never fetched again and expire naturally within 60 seconds. Invalidation cost is O(1).

```
// On CreateProduct or UpdateProduct:
await _cache.IncrementVersionAsync(ProductCacheKeys.ListVersionKey);

// On GetProductList:
var version = await _cache.GetVersionAsync(ProductCacheKeys.ListVersionKey);
var key = $"products:list:v{version}:{SHA256(query)[..16]}";
var cached = await _cache.GetAsync<PagedResult>(key);
if (cached is not null) return cached;
// ... query DB, store in cache with 60s TTL
```

Single-product cache invalidation

On `UpdateProduct`, `AdjustStock`, or `UpdateProductItem`: `cache.RemoveAsync("product:{id}")` is called immediately after the database write. The next read repopulates the cache from the database with a fresh 10-minute TTL.

5.2 Concurrency Control

Products — PostgreSQL xmin + ETag/If-Match

PostgreSQL's `xmin` system column is a free, storage-level row version token incremented by Postgres on every UPDATE — no application-managed version column needed. EF Core maps it as a shadow concurrency token via `IsRowVersion()`.

```
// ProductRepository - set expected version before update
_db.Entry(product).Property("xmin").OriginalValue = expectedXmin;
// EF Core issues: UPDATE products SET ... WHERE id = @id AND xmin = @expectedXmin
// Zero rows affected → DbUpdateConcurrencyException → 409 Conflict
```

ProductItems — application-managed integer Version

Each `ProductItem` carries an integer `Version` field incremented by the domain method `UpdateDetails()`. The `PATCH /product-items/{id}` endpoint requires an `If-Match` header containing the expected version number. The handler compares before saving; a mismatch throws `ConcurrencyConflictException` → 409.

Stock adjustment — atomic conditional SQL UPDATE

Rather than the load-then-save pattern (which has a race window between the read and write), stock adjustment is a single atomic SQL statement:

```
-- For a decrement of N units:
UPDATE product_items
SET qty_in_stock = qty_in_stock - @delta
WHERE id = @itemId AND qty_in_stock ≥ @delta -- guard: prevents negative stock
RETURNING qty_in_stock, (old qty_in_stock) AS available_quantity
```

If the guard fails (insufficient stock), zero rows are updated and the handler returns `409 Conflict` with the current `availableQuantity` in the response body. Because this is a single database round-trip with no intermediate read, 30 concurrent decrements against a stock of 10 will result in exactly 10 successes and 20 failures — verified by the `StockConcurrencyTests` integration test.

5.3 Database Query Optimisation

Technique	Where applied	Benefit
GIN index on <code>search_vector</code>	Product FTS queries	O(log n) ranked search vs. O(n) sequential scan
GIN trigram index on <code>name</code>	Fuzzy fallback search	Similarity queries without full table scan

Technique	Where applied	Benefit
GIN index on <code>attributes</code> (<code>jsonb_path_ops</code>)	Attribute filter queries	Efficient <code>@></code> containment checks on jsonb
Cursor pagination	Product list	$O(\log n)$ per page via index seek; no row-skip overhead
Eager loading with <code>Include().ThenInclude()</code>	<code>GetProduct</code> , <code>ListProducts</code>	Single SQL join instead of $N+1$ selects
Partial filtered index on <code>product_items</code>	Active in-stock items	Index covers only <code>is_active=true AND qty_in_stock>0</code> rows
Batch insert (1 000 rows per <code>SaveChanges</code>)	<code>DbInitializer</code> seeder	Avoids 5 000 individual INSERT round-trips
<code>ChangeTracker.AutoDetectChangesEnabled = false</code>	Seeder bulk insert	Removes $O(n^2)$ change detection during batch
<code>ExecuteUpdateAsync</code> (bulk UPDATE)	Stock adjustment	No entity load — single parameterised UPDATE
<code>citext</code> extension on slug columns	Slug uniqueness checks	Case-insensitive unique index with no function overhead

The `StockConcurrencyTests` integration test fires 30 concurrent HTTP requests against a product item with 10 units in stock. It asserts exactly 10 successes, exactly 20 failures, and a final stock of 0 — proving the atomic SQL guard prevents overselling under real concurrent load.